

详解 pd.concat() 中 axis=0 和 axis=1 的区别

核心概念

- **axis=0**：纵向拼接（按行拼接，上下堆叠）
- **axis=1**：横向拼接（按列拼接，左右并排）

详细对比

axis=0：纵向拼接（默认）

代码块

```
1  import pandas as pd
2  import numpy as np
3
4  # 创建两个示例 DataFrame
5  df1 = pd.DataFrame({
6      '姓名': ['张三', '李四'],
7      '年龄': [25, 30],
8      '城市': ['北京', '上海']
9  })
10
11  df2 = pd.DataFrame({
12      '姓名': ['王五', '赵六'],
13      '年龄': [28, 35],
14      '城市': ['广州', '深圳']
15  })
16
17  print("df1:")
18  print(df1)
19  print("\ndf2:")
20  print(df2)
21
22  # axis=0: 纵向拼接（上下堆叠）
23  result_axis0 = pd.concat([df1, df2], axis=0)
24  print("\naxis=0 纵向拼接结果（行数增加）：")
```

```
25 print(result_axis0)
26 print(f"形状: {result_axis0.shape}") # (4, 3) - 4行3列
```

输出效果：

代码块

```
1 df1:
2   姓名  年龄  城市
3 0 张三  25  北京
4 1 李四  30  上海
5
6 df2:
7   姓名  年龄  城市
8 0 王五  28  广州
9 1 赵六  35  深圳
10
11 axis=0 纵向拼接结果（行数增加）：
12   姓名  年龄  城市
13 0 张三  25  北京
14 1 李四  30  上海
15 0 王五  28  广州 ← 索引可能重复
16 1 赵六  35  深圳
17
18 形状：(4, 3)
```

2 axis=1：横向拼接

代码块

```
1 # 创建两个不同的 DataFrame
2 df1 = pd.DataFrame({
3     '姓名': ['张三', '李四', '王五'],
4     '年龄': [25, 30, 28]
5 })
6
7 df2 = pd.DataFrame({
```

```

8     '城市': ['北京', '上海', '广州'],
9     '收入': [8000, 9000, 7500]
10 })
11
12 print("df1:")
13 print(df1)
14 print("\ndf2:")
15 print(df2)
16
17 # axis=1: 横向拼接 (左右并排)
18 result_axis1 = pd.concat([df1, df2], axis=1)
19 print("\naxis=1 横向拼接结果 (列数增加) :")
20 print(result_axis1)
21 print(f"形状: {result_axis1.shape}") # (3, 4) - 3行4列

```

输出效果：

代码块

```

1  df1:
2      姓名  年龄
3  0  张三   25
4  1  李四   30
5  2  王五   28
6
7  df2:
8      城市  收入
9  0  北京  8000
10 1  上海  9000
11 2  广州  7500
12
13 axis=1 横向拼接结果 (列数增加) :
14      姓名  年龄  城市  收入
15 0  张三   25  北京  8000
16 1  李四   30  上海  9000
17 2  王五   28  广州  7500
18
19 形状: (3, 4)

```

完整对比示例

代码块

```
1  import pandas as pd
2  import numpy as np
3
4  print("="*70)
5  print("pd.concat() axis参数对比演示")
6  print("="*70)
7
8  # ===== 场景1: axis=0 纵向拼接 =====
9  print("\n【场景1】axis=0 - 纵向拼接（合并多个数据源的同结构数据）")
10 print("-"*70)
11
12 df_jan = pd.DataFrame({
13     '日期': ['2024-01-01', '2024-01-02'],
14     '销售额': [1000, 1500],
15     '订单数': [10, 15]
16 })
17
18 df_feb = pd.DataFrame({
19     '日期': ['2024-02-01', '2024-02-02'],
20     '销售额': [1200, 1800],
21     '订单数': [12, 18]
22 })
23
24 df_mar = pd.DataFrame({
25     '日期': ['2024-03-01', '2024-03-02'],
26     '销售额': [1400, 2000],
27     '订单数': [14, 20]
28 })
29
30 print("1月份数据:")
31 print(df_jan)
32 print("\n2月份数据:")
33 print(df_feb)
34 print("\n3月份数据:")
35 print(df_mar)
36
37 # 纵向拼接三个月的数据
38 df_quarter = pd.concat([df_jan, df_feb, df_mar], axis=0, ignore_index=True)
39 print("\n第一季度数据（纵向拼接后）:")
40 print(df_quarter)
41 print(f"形状: {df_quarter.shape} ← 行数从2增加到6")
42
```

```

43 # ===== 场景2: axis=1 横向拼接 =====
44 print("\n\n【场景2】axis=1 - 横向拼接（为数据添加新特征/列）")
45 print("-"*70)
46
47 df_base = pd.DataFrame({
48     '学生': ['张三', '李四', '王五'],
49     '学号': [101, 102, 103]
50 })
51
52 df_scores = pd.DataFrame({
53     '数学': [90, 85, 92],
54     '英语': [88, 90, 87]
55 })
56
57 df_info = pd.DataFrame({
58     '班级': ['一班', '二班', '一班'],
59     '性别': ['男', '女', '男']
60 })
61
62 print("基础信息:")
63 print(df_base)
64 print("\n成绩数据:")
65 print(df_scores)
66 print("\n其他信息:")
67 print(df_info)
68
69 # 横向拼接, 整合所有信息
70 df_complete = pd.concat([df_base, df_scores, df_info], axis=1)
71 print("\n完整学生信息（横向拼接后）:")
72 print(df_complete)
73 print(f"形状: {df_complete.shape} ← 列数从2增加到6")

```

⚠ 注意事项和常见问题

1. 索引处理

代码块

```
1 df1 = pd.DataFrame({'A': [1, 2]}, index=[0, 1])
```

```

2  df2 = pd.DataFrame({'A': [3, 4]}, index=[0, 1])
3
4  # 默认保留原索引 (可能导致索引重复)
5  result_default = pd.concat([df1, df2], axis=0)
6  print("默认索引 (有重复) :")
7  print(result_default)
8  #    A
9  # 0  1
10 # 1  2
11 # 0  3 ← 索引/重复
12 # 1  4
13
14 # 使用 ignore_index 重置索引
15 result_reset = pd.concat([df1, df2], axis=0, ignore_index=True)
16 print("\n重置索引:")
17 print(result_reset)
18 #    A
19 # 0  1
20 # 1  2
21 # 2  3 ← 索引/连续
22 # 3  4

```

2. 列名不一致时的处理

代码块

```

1  df1 = pd.DataFrame({
2      '姓名': ['张三', '李四'],
3      '年龄': [25, 30]
4  })
5
6  df2 = pd.DataFrame({
7      '姓名': ['王五', '赵六'],
8      '城市': ['北京', '上海'] # 列名不同
9  })
10
11 # axis=0 时, 缺失的列用 NaN 填充
12 result = pd.concat([df1, df2], axis=0, ignore_index=True)
13 print("列名不一致时的纵向拼接:")
14 print(result)
15 #    姓名    年龄    城市

```

```
16 # 0 张三 25.0 NaN
17 # 1 李四 30.0 NaN
18 # 2 王五 NaN 北京
19 # 3 赵六 NaN 上海
```

3. 行数不一致时的横向拼接

代码块

```
1 df1 = pd.DataFrame({
2     'A': [1, 2, 3]
3 })
4
5 df2 = pd.DataFrame({
6     'B': [4, 5] # 少一行
7 })
8
9 # axis=1 时, 缺失的行用 NaN 填充
10 result = pd.concat([df1, df2], axis=1)
11 print("行数不一致时的横向拼接:")
12 print(result)
13 #    A    B
14 # 0  1  4.0
15 # 1  2  5.0
16 # 2  3  NaN ← 自动填充NaN
```

常用参数总结

代码块

```
1 pd.concat(
2     objs, # 要拼接的DataFrame列表
3     axis=0, # 0=纵向, 1=横向
4     ignore_index=False, # 是否重置索引
5     join='outer', # outer=并集(默认), inner=交集
```

```
6     keys=None,          # 添加层级索引标识来源
7     sort=False         # 是否对列名排序
8 )
```

实际应用示例：

代码块

```
1  # 1. 纵向拼接 + 重置索引
2  df_all = pd.concat([df1, df2, df3], axis=0, ignore_index=True)
3
4  # 2. 横向拼接 + 只保留共同的行 (inner join)
5  df_merged = pd.concat([df1, df2], axis=1, join='inner')
6
7  # 3. 添加来源标识
8  df_labeled = pd.concat([df1, df2], keys=['来源1', '来源2'])
```

选择指南

场景	使用	说明
合并多个月份的销售数据	<code>axis=0</code>	行增加，列不变
为学生表添加成绩列	<code>axis=1</code>	列增加，行不变
整合多个CSV文件	<code>axis=0</code>	相同结构的数据堆叠
为数据添加新特征	<code>axis=1</code>	基于索引对齐添加列
时间序列数据扩展	<code>axis=0</code>	追加新的时间点

记忆口诀：

- `axis=0` → 沿着行的方向（0轴）→ **上下拼接** → 行变多
- `axis=1` → 沿着列的方向（1轴）→ **左右拼接** → 列变多